

**IMPLEMENTASI SISTEM POS BERBASIS WEB UNTUK
OPTIMALISASI EFISIENSI OPERASIONAL PADA USAHA
LAUNDRY**

LAPORAN PROYEK II

Diajukan untuk Memenuhi Kelulusan Matakuliah Proyek 2 pada Program Studi D-IV
Teknik Informatika



DISUSUN OLEH :

Alifya Azzahra 714240011

Ismi Nabilah 714240056

**PROGRAM STUDI D-IV TEKNIK INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS LOGISTIK DAN BISNIS INTERNASIONAL
BANDUNG**

2026

LEMBAR PENGESAHAN

IMPLEMENTASI SISTEM POS BERBASIS WEB UNTUK OPTIMALISASI EFISIENSI OPERASIONAL PADA USAHA LAUNDRY

Diajukan untuk Memenuhi Kelulusan Matakuliah Proyek 2
Program Studi D-IV Teknik Informatika

Disusun Oleh:

Alifya Azzahra 714240011

Ismi Nabilah 714240056

DOSEN PEMBIMBING

KOORDINATOR PROYEK 2

Rolly Maulana Awangga,
S.T., MT., CAIP., SFPC
NIK : 117.86.219

Rd. Nuraini Siti Fathonah,
S.S., M.Hum., SFPC
NIK : 118.72.251

MENYETUJUI,
KETUA PROGRAM STUDI DIV TEKNIK INFORMATIKA

Roni Andarsyah, S.T., M.Kom., SFPC
NIK. 115.88.193

SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME

Nama : Ismi Nabilah

NPM : 714240056

Program Studi : D-IV Teknik Informatika

Judul : Implementasi Sistem POS Berbasis Web untuk Optimalisasi

Efisiensi Operasional pada Usaha Laundry

Menyatakan bahwa:

1. Proyek Pemrograman aplikasi (Proyek 2) saya ini adalah asli dan belum pernah diajukan untuk memenuhi kelulusan proyek 2 pada program studi D-IV Teknik informatika baik di Universitas Logistik Bisnis Internasional maupun di perguruan tinggi lainnya.
2. Proyek pemrograman aplikasi (Proyek 2) ini adalah murni gagasan, rumusan, dan peneliti saya sendiri tanpa bantuan orang lain, kecuali arahan pembimbing.
3. Dalam proyek pemrograman aplikasi (Proyek 2) ini tidak terdapat karya atau pendapat yang telah ditulis ataupun dipublikasi orang lain, kecuali secara tertulis dengan jelas dicantumkan sebagai acuan dalam naskah dengan disebutkan nama pengarang dan dicantumkan dalam daftar pustaka.
4. Pernyataan ini saya buat dengan sesungguhnya dan apabila di kemudian hari terdapat penyimpangan-penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik berupa pencabutan gelar yang telah diperoleh karena karya ini, serta sanksi lainnya sesuai dengan norma yang berlaku di perguruan tinggi lain.

Bandung, 2026

Yang Membuat Pernyataan,

Ismi Nabilah

NIM. 714240056

SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME

Nama : Alifya Azzahra

NPM : 714240011

Program Studi : D-IV Teknik Informatika

Judul : Implementasi Sistem POS Berbasis Web untuk Optimalisasi

Efisiensi Operasional pada Usaha Laundry

Menyatakan bahwa:

1. Proyek Pemrograman aplikasi (Proyek 2) saya ini adalah asli dan belum pernah diajukan untuk memenuhi kelulusan proyek 2 pada program studi D-IV Teknik informatika baik di Universitas Logistik Bisnis Internasional maupun di perguruan tinggi lainnya.
2. Proyek Pemrograman aplikasi (Proyek 2) ini adalah murni gagasan, rumusan, dan peneliti saya sendiri tanpa bantuan orang lain, kecuali arahan pembimbing.
3. Dalam proyek Pemrograman aplikasi (Proyek 2) ini tidak terdapat karya atau pendapat yang telah ditulis ataupun dipublikasi orang lain, kecuali secara tertulis dengan jelas dicantumkan sebagai acuan dalam naskah dengan disebutkan nama pengarang dan dicantumkan dalam daftar pustaka.
4. Pernyataan ini saya buat dengan sesungguhnya dan apabila di kemudian hari terdapat penyimpangan-penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik berupa pencabutan gelar yang telah diperoleh karena karya ini, serta sanksi lainnya sesuai dengan norma yang berlaku di perguruan tinggi lain.

Bandung, 2026

Yang Membuat Pernyataan,

Alifya Azzahra

NIM. 714240011

ABSTRAK

Pencatatan transaksi pada operasional laundry yang masih bersifat manual seringkali menimbulkan risiko ketidakakuratan data dan kurangnya transparansi pembayaran. Penelitian ini bertujuan untuk mengembangkan Sistem *Point of Sale* (POS) WellClean Laundry berbasis web yang mengintegrasikan fitur manajemen pelanggan, layanan, dan transaksi otomatis. Sistem ini dibangun menggunakan bahasa pemrograman Go dengan arsitektur yang mendukung efisiensi pemrosesan data. Salah satu fitur unggulan yang diimplementasikan adalah integrasi pembayaran digital melalui QRIS yang didukung oleh mekanisme *webhook* untuk mendeteksi dana masuk secara *real-time* dan menutup jendela transaksi secara otomatis. Guna menjamin keandalan sistem, dilakukan audit pengujian unit (*unit testing*) menggunakan *tool go tool cover*. Hasil pengujian menunjukkan fungsionalitas krusial seperti *GopayHandler* dan *HashPassword* mencapai tingkat validitas sebesar 100.0%, dengan rata-rata keseluruhan *statements* sistem sebesar 15.3%. Implementasi ini terbukti dapat meningkatkan akurasi operasional dan memberikan kemudahan bagi kasir dalam memproses transaksi digital.

Kata Kunci: *Point of Sale, Go, QRIS, Unit Testing, Code Coverage.*

ABSTRAK

Manual transaction recording in laundry operations often leads to risks of data inaccuracy and a lack of payment transparency. This study aims to develop a web-based Point of Sale (POS) system for WellClean Laundry that integrates customer management, service management, and automated transaction features. The system is built using the Go programming language with an architecture designed for efficient data processing. One of the key features implemented is the integration of digital payments via QRIS, supported by a webhook mechanism to detect incoming funds in real-time and automatically close the transaction modal. To ensure system reliability, a unit testing audit was conducted using the go tool cover tool. The test results show that crucial functionalities such as GopayHandler and HashPassword achieved a 100.0

Keywords: *Point of Sale, Go, QRIS, Unit Testing, Code Coverage.*

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat-Nya sehingga Laporan Proyek 2 yang berjudul "Implementasi Sistem POS Berbasis Web untuk Optimalisasi Efisiensi Operasional pada Usaha Laundry" dapat diselesaikan tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu syarat kelulusan mata kuliah Proyek 2 pada Program Studi D4 Teknik Informatika, Universitas Logistik dan Bisnis Internasional.

Isi laporan ini memaparkan perancangan, pengembangan, serta implementasi sistem POS yang digunakan untuk membantu operasional WellClean Laundry dalam mengelola data pelanggan, layanan, serta transaksi pembayaran secara digital. Harapannya, platform ini dapat membantu meningkatkan kualitas pelayanan, transparansi pembayaran melalui QRIS, serta mendukung proses pencatatan transaksi agar lebih tertata dan akurat.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran yang membangun dari dosen pembimbing maupun pembaca sangat diharapkan demi penyempurnaan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menjadi salah satu referensi dalam pengembangan sistem informasi di bidang layanan jasa laundry.

Bandung, 2026

DAFTAR ISI

LEMBAR PENGESAHAN	i
SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME	ii
SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME	iii
ABSTRAK	iv
ABSTRACT	v
KATA PENGANTAR	vi
DAFTAR ISI	vii
DAFTAR GAMBAR	ix
DAFTAR TABEL	x
BAB I PENDAHULUAN	1
1.1 Deskripsi Aplikasi	1
1.2 Identifikasi Masalah	2
1.3 Tujuan	2
1.4 Lingkup Dokumentasi	3
BAB II LANDASAN TEORI	4
2.1 Sistem Informasi	4
2.2 Point of Sale (POS)	4
2.3 Usaha Laundry	4
2.4 Usability dan User Experience	5
BAB III ANALISIS PERANCANGAN	6
3.1 Analisis	7
3.1.1 Analisis Fungsi Fungsional	7
3.1.2 Analisis Non Fungsional	7
3.2 Perancangan	8
3.2.1 Usecase Diagram	8
3.2.2 Antarmuka (UI)	9
3.2.3 Struktur Database	10
3.2.4 Algoritma fungsi	11

3.2.5	Flowchart	11
BAB IV	IMPLEMENTASI DAN PENGUJIAN	13
4.1	Pembahasan Hasil Implementasi	13
4.1.1	Implementasi Antarmuka Pengguna (User Interface)	13
4.1.2	Implementasi Logika Program (Bedah Kode)	16
4.2	Pengujian dan Hasil Pengujian	51
4.2.1	Pengujian Unit (Unit Testing)	51
4.2.2	Hasil Pengujian Antarmuka dan Alur Fungsional	52
BAB V	KESIMPULAN DAN SARAN	58
5.1	Kesimpulan	58
5.2	Saran	58
	DAFTAR PUSTAKA	60
	LAMPIRAN	61
	GLOSARIUM	63

DAFTAR GAMBAR

III.1 Prototyping	6
III.2 Usecase Diagram	8
III.3 Struktur Database	10
III.4 Flowchart	11
IV.5 Halaman Otentikasi (Login)	13
IV.6 Halaman Dashboard	14
IV.7 Halaman Manajemen Pelanggan	14
IV.8 Halaman Manajemen Transaksi	15
IV.9 Halaman Laporan	15
IV.10 Halaman Kasir	16
IV.11 Hasil Pengujian Code Coverage	52
IV.12 Hasil Pengujian Login	53
IV.13 Hasil Pengujian Dashboard	53
IV.14 Hasil Pengujian Pelanggan	54
IV.15 Hasil Pengujian Layanan	54
IV.16 Hasil Pengujian POS Mode Kasir	55
IV.17 Hasil Pengujian Notifikasi Payment	55
IV.18 Hasil Pengujian Laporan	56
IV.19 Hasil Pengujian Laporan Transaksi Harian	56
IV.20 Hasil Pengujian Laporan Keuangan Harian	56
L.1 Rekapitulasi Cakupan Baris Kode (Statements)	61
L.2 Tangkapan Layar Hasil Codecoverage	62

DAFTAR TABEL

BAB I

PENDAHULUAN

1.1 Deskripsi Aplikasi

Perkembangan teknologi informasi mendorong usaha kecil, termasuk laundry, untuk beralih dari pencatatan manual ke sistem digital. Salah satu solusi yang dikembangkan adalah Sistem Point of Sale (POS) Laundry berbasis web, yang berfungsi untuk mencatat transaksi, mengelola data pelanggan, memantau status cucian, serta menyusun laporan operasional secara terintegrasi. Dengan berbasis web, sistem ini dapat diakses secara fleksibel dari berbagai perangkat, sehingga meningkatkan efisiensi dan memudahkan pemilik usaha dalam melakukan monitoring (Achyani, Aulianita & Mukhayaroh, 2024).

Penelitian sebelumnya menegaskan bahwa aplikasi laundry berbasis web mampu menyederhanakan proses pencarian dan pengelolaan layanan laundry, sekaligus meningkatkan transparansi data dan akurasi pencatatan (Singh & Dhaiya, 2022). Selain itu, pendekatan berbasis pengguna dalam pengembangan sistem laundry terbukti dapat memperbaiki alur operasional dan meningkatkan kepuasan pelanggan, terutama melalui antarmuka yang mudah digunakan dan proses transaksi yang lebih cepat (Kalo & Amron, 2023).

Studi lokal juga menunjukkan bahwa sistem informasi laundry berbasis web dapat membantu pemilik usaha dalam mengurangi kesalahan pencatatan, mempercepat proses transaksi, serta menghasilkan laporan yang lebih rapi dan terstruktur (Sugiharto et al., 2023). Penelitian lain menekankan bahwa penerapan sistem digital pada usaha kecil menengah (UMKM) berpengaruh positif terhadap efisiensi operasional dan kualitas layanan, sehingga mendukung keberlanjutan bisnis (Maulana, Kusumah & Kamilah, 2025).

Berdasarkan kondisi tersebut, usaha laundry yang masih bergantung pada pencatatan manual berisiko menghadapi kendala seperti kesalahan hitung, keterlambatan informasi, dan pengelolaan data yang kurang optimal. Oleh karena itu, pengembangan Sistem POS Laundry Berbasis Website diharapkan menjadi solusi yang mampu meningkatkan efisiensi operasional, kualitas layanan, serta kepuasan pelanggan secara berkelanjutan.

1.2 Identifikasi Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka permasalahan yang diidentifikasi dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana sistem POS Laundry berbasis web dapat menggantikan pencatatan manual yang masih digunakan?
2. Bagaimana sistem POS Laundry berbasis web dapat membantu pemilik dalam mengelola transaksi, pelanggan, dan laporan secara efisien?
3. Bagaimana antarmuka sistem POS Laundry berbasis web dapat dirancang agar mudah digunakan oleh karyawan maupun pemilik?
4. Bagaimana sistem POS Laundry berbasis web dapat meningkatkan akurasi pencatatan dan mengurangi kesalahan transaksi?

Rumusan masalah ini sejalan dengan penelitian terdahulu yang menekankan pentingnya digitalisasi layanan laundry untuk meningkatkan efisiensi, akurasi pencatatan, serta kepuasan pengguna (Achyani, Aulianita & Mukhayaroh, 2024; Mutia et al., 2025; Kalo & Amron, 2023; Maulana, Kusumah & Kamilah, 2025; Sugiharto et al., 2023).

1.3 Tujuan

Tujuan dari penelitian dan pengembangan sistem POS Laundry berbasis web ini adalah:

1. Merancang dan membangun sistem POS Laundry berbasis web untuk menggantikan pencatatan manual.
2. Menyediakan fitur transaksi, manajemen pelanggan, dan laporan yang terintegrasi.
3. Meningkatkan efisiensi operasional laundry melalui sistem digital yang cepat dan stabil.
4. Mengurangi kesalahan pencatatan serta meningkatkan akurasi data transaksi.

Tujuan ini didukung oleh penelitian sebelumnya yang menunjukkan bahwa sistem berbasis web dapat meningkatkan efisiensi operasional, memperbaiki kualitas layanan, dan mendukung keberlanjutan usaha laundry (Achyani, Aulianita & Mukhayaroh, 2024; Kalo & Amron, 2023; Sugiharto et al., 2023; Maulana, Kusumah & Kamilah, 2025).

1.4 Lingkup Dokumentasi

Lingkup dokumentasi pada proyek ini dibatasi pada aspek-aspek utama yang mendukung pengembangan Sistem POS Laundry berbasis web, meliputi:

1. Analisis kebutuhan sistem yang mencakup identifikasi modul inti seperti transaksi, pelanggan, dan laporan.
2. Perancangan sistem berbasis web yang meliputi desain antarmuka sederhana, responsif, dan mudah digunakan.
3. Implementasi modul inti berupa transaksi laundry, manajemen pelanggan, serta laporan operasional dan keuangan.
4. Evaluasi awal sistem melalui uji coba terbatas untuk menilai efisiensi dan akurasi pencatatan.

Batasan dokumentasi tidak mencakup pengembangan aplikasi mobile, integrasi notifikasi otomatis, maupun fitur tambahan di luar modul inti. Fokus diarahkan pada sistem berbasis web yang mendukung pencatatan transaksi, pemantauan status cucian, dan penyusunan laporan secara terstruktur (Mutia et al., 2025; Lizal et al., 2025; Kalo & Amron, 2023).

BAB II

LANDASAN TEORI

2.1 Sistem Informasi

Sistem informasi merupakan suatu kombinasi dari teknologi, manusia, dan prosedur yang digunakan untuk mengumpulkan, mengolah, menyimpan, dan menyajikan informasi guna mendukung kegiatan operasional dan pengambilan keputusan dalam suatu organisasi. Dalam konteks usaha jasa, sistem informasi berperan penting dalam meningkatkan efisiensi operasional, akurasi pencatatan, serta transparansi data.

Pada era digital, sistem informasi berbasis web banyak digunakan karena mampu menyediakan akses data secara fleksibel dan terintegrasi. Penelitian menunjukkan bahwa penerapan sistem informasi berbasis web pada layanan laundry mampu mengurangi kesalahan pencatatan, mempercepat proses transaksi, serta meningkatkan efektivitas pengelolaan data operasional (Achyani, Aulianita & Mukhayaroh, 2024; Mutia et al., 2025; Maulana, Kusumah & Kamilah, 2025).

2.2 Point of Sale (POS)

Point of Sale (POS) merupakan sistem yang digunakan untuk mencatat dan mengelola transaksi penjualan secara digital. Sistem POS tidak hanya berfungsi sebagai alat pencatatan transaksi, tetapi juga terintegrasi dengan pengelolaan data pelanggan, laporan keuangan, dan pemantauan status layanan.

POS berbasis web memiliki keunggulan dibandingkan sistem konvensional karena memungkinkan akses multi perangkat, pemrosesan data secara real time, serta kemudahan monitoring oleh pemilik usaha. Penelitian sebelumnya membuktikan bahwa penggunaan sistem POS berbasis web pada usaha laundry dapat meningkatkan akurasi pencatatan transaksi dan efisiensi operasional secara signifikan (Singh & Dhaiya, 2022; Kalo & Amron, 2023).

2.3 Usaha Laundry

Usaha laundry sebagai bagian dari Usaha Mikro, Kecil, dan Menengah (UMKM) sering menghadapi permasalahan dalam pencatatan manual, seperti kesalahan transaksi, kehilangan data, dan keterlambatan penyusunan laporan. Kondisi tersebut berdampak pada rendahnya efisiensi operasional dan kualitas layanan kepada pelanggan.

Digitalisasi usaha laundry melalui penerapan sistem POS berbasis web menjadi solusi untuk mengatasi permasalahan tersebut. Berbagai penelitian menunjukkan bahwa sistem informasi laundry berbasis web mampu menyederhanakan proses pencatatan, mempercepat transaksi, serta meningkatkan kepuasan pelanggan melalui antarmuka yang mudah digunakan (Sugiharto et al., 2023; Achyani, Aulianita & Mukhayaroh, 2024; Mutia et al., 2025). Selain itu, digitalisasi laundry juga mendukung keberlanjutan usaha dengan menyediakan laporan operasional yang lebih rapi dan terstruktur (Lizal et al., 2025; Maulana, Kusumah & Kamilah, 2025).

2.4 Usability dan User Experience

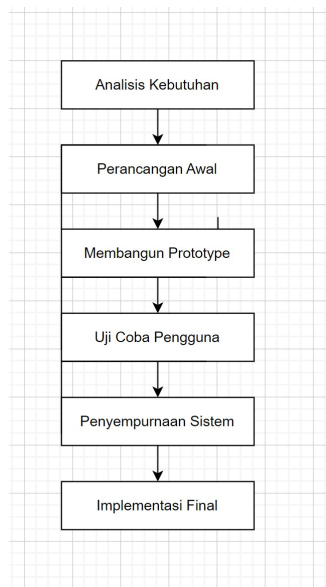
Usability dan user experience merupakan aspek penting dalam pengembangan sistem berbasis web, khususnya pada sistem POS Laundry yang digunakan secara rutin oleh karyawan dan pemilik usaha. Sistem yang memiliki antarmuka sederhana, informatif, dan mudah dipahami akan mempermudah proses operasional serta mengurangi kesalahan penggunaan.

Penelitian terdahulu menegaskan bahwa pendekatan berbasis pengguna dalam pengembangan sistem laundry berbasis web berkontribusi pada peningkatan kepuasan pengguna dan efektivitas penggunaan sistem (Kalo & Amron, 2023; Singh & Dhaiya, 2022). Oleh karena itu, penerapan prinsip usability menjadi landasan penting dalam perancangan sistem POS Laundry berbasis web agar sistem dapat digunakan secara optimal dan berkelanjutan.

BAB III

ANALISIS PERANCANGAN

Penelitian ini menggunakan metode Prototyping, yaitu pendekatan pengembangan sistem yang dilakukan secara iteratif dengan berfokus pada pembuatan versi awal aplikasi (prototype) yang dapat langsung diuji oleh pengguna. Metode ini dipilih karena memungkinkan peneliti memahami kebutuhan pengguna secara lebih mendalam melalui interaksi nyata dengan sistem yang sedang dikembangkan. Dengan adanya prototype, pengguna dapat memberikan masukan konkret terhadap fitur, tampilan, dan alur kerja sistem sehingga proses pengembangan menjadi lebih adaptif dan responsif terhadap kebutuhan riil di lapangan. Objek penelitian adalah sistem Point of Sales (POS) Laundry berbasis web yang dirancang untuk membantu proses pencatatan transaksi, pengelolaan data pelanggan, serta pembuatan laporan keuangan secara otomatis. Adapun subjek penelitian terdiri dari dua peran utama dalam operasional laundry, yaitu pemilik usaha dan kasir. Pemilik berperan sebagai pengambil keputusan sekaligus pengelola sistem secara keseluruhan, sedangkan kasir bertugas menjalankan transaksi harian dan berinteraksi langsung dengan pelanggan.



Gambar III.1: Prototyping

3.1 Analisis

3.1.1 Analisis Fungsi Fungsional

Analisis fungsional dilakukan untuk mengidentifikasi kebutuhan utama sistem POS Laundry berbasis web. Sistem ini dirancang agar dapat menggantikan pencatatan manual dengan menyediakan fitur inti sebagai berikut:

- Transaksi Laundry: mencatat pesanan pelanggan, jenis layanan (cuci, setrika, paket), jumlah, harga, dan status cucian.
- Manajemen Pelanggan: menyimpan data pelanggan, riwayat transaksi, serta mempermudah pencarian informasi.
- Laporan Operasional dan Keuangan: menghasilkan laporan harian, mingguan, dan bulanan terkait jumlah transaksi, pendapatan, serta status cucian.
- Pengelolaan Status Cucian: menampilkan status cucian (proses, selesai, diambil) secara real time.

Fungsi fungsi ini mendukung tujuan sistem untuk meningkatkan efisiensi, akurasi pencatatan, dan transparansi layanan.

3.1.2 Analisis Non Fungsional

Selain kebutuhan fungsional, sistem juga harus memenuhi kebutuhan non fungsional agar dapat digunakan secara optimal:

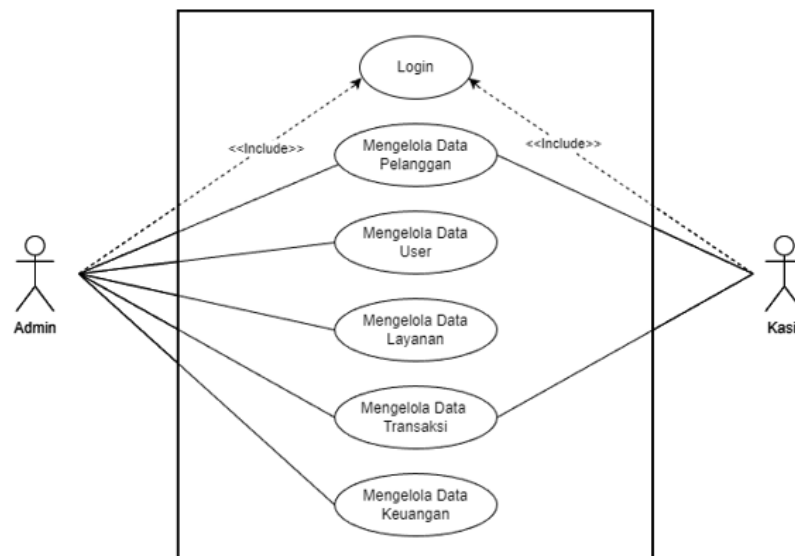
- Keamanan Data: sistem harus melindungi data pelanggan dan transaksi dari akses tidak sah.
- Kinerja Sistem: aplikasi harus mampu memproses transaksi dengan cepat dan stabil.
- Usability: antarmuka harus sederhana, responsif, dan mudah dipahami oleh karyawan maupun pemilik.
- Portabilitas: sistem berbasis web dapat diakses melalui berbagai perangkat (PC, laptop, smartphone).
- Reliabilitas: sistem harus mampu beroperasi secara konsisten tanpa gangguan yang signifikan.

3.2 Perancangan

Tahap perancangan merupakan proses penerjemahan hasil analisis kebutuhan ke dalam bentuk rancangan teknis yang akan digunakan sebagai acuan dalam pengembangan sistem POS Laundry berbasis web. Perancangan ini mencakup struktur menu, desain antarmuka pengguna, perancangan basis data, serta logika fungsi yang mendukung operasional laundry secara digital.

Tujuan utama dari perancangan adalah memastikan bahwa sistem yang dibangun sesuai dengan kebutuhan fungsional dan non fungsional yang telah diidentifikasi sebelumnya. Dengan rancangan yang terstruktur, pengembangan sistem dapat dilakukan secara efisien, terarah, dan minim risiko kesalahan implementasi. Setiap komponen yang dirancang baik dari sisi tampilan maupun proses internal harus mampu mendukung pencatatan transaksi, pengelolaan pelanggan, dan penyusunan laporan secara akurat dan real time.

3.2.1 Usecase Diagram



Gambar III.2: Usecase Diagram

Berdasarkan gambar 3.2.1 diatas, struktur Menu yang tersedia pada aplikasi adalah:

1. Admin:
 - (a) Login
 - (b) Mengelola Data Pelanggan

- (c) Mengelola Data User
- (d) Mengelola Data Layanan
- (e) Mengelola Data Transaksi
- (f) Mengelola Data Keuangan

2. Kasir:

- (a) Login
- (b) Mengelola Data Pelanggan
- (c) Mengelola Data Transaksi

3.2.2 Antarmuka (UI)

Desain antarmuka yang dirancang menggunakan prinsip responsif dan sederhana agar mudah digunakan oleh pengguna, baik melalui komputer maupun smartphone. Berdasarkan analisis kebutuhan, berikut adalah komponen utama dari antarmuka yang dirancang:

1. Halaman Login: Proses login melibatkan pengisian formulir terautentikasi untuk mendapatkan akses ke sistem yang memisahkan administrator dan pengguna.
2. Dashboard Admin: Area admin utama dengan menu navigasi untuk melihat data pengguna, data laundry, dan ringkasan laporan keuangan serta kinerja bisnis.
3. Dashboard Kasir: Ruang kerja utama untuk Kasir yang menyediakan fitur cepat untuk memasukkan transaksi baru, memeriksa status pelanggan, dan menganalisis data pelanggan.
4. Halaman Transaksi: Antarmuka untuk menentukan jenis layanan laundry (kiloan/satuan), menghitung total biaya secara otomatis, dan memproses pembayaran.
5. truk Transaksi: Tampilan bukti pembayaran yang dapat dicetak atau dikirim secara digital kepada pelanggan berisi detail layanan dan total biaya.

3.2.3 Struktur Database



Gambar III.3: Struktur Database

Berdasarkan gambar 3.2.3 diatas, sistem ini menggunakan tiga tabel utama yang saling berelasi untuk mengelola data operasional laundry:

1. Tabel Pelanggan. Digunakan untuk menyimpan informasi identitas pelanggan seperti nama, alamat, telepon.
2. Tabel Layanan. Berisi daftar jenis jasa laundry yang ditawarkan, termasuk informasi seperti nama layanan, harga, satuan (misalnya per kg atau per potong), dan deskripsi.
3. Tabel Transaksi. Mencatat setiap pesanan yang dilakukan oleh pelanggan, termasuk total (biaya), status (status pengerjaan/pembayaran), dan berat (untuk laundry kiloan).

Selain tabel utama, sistem mengimplementasikan sebuah Database View bernama "transaksi_wi"b.

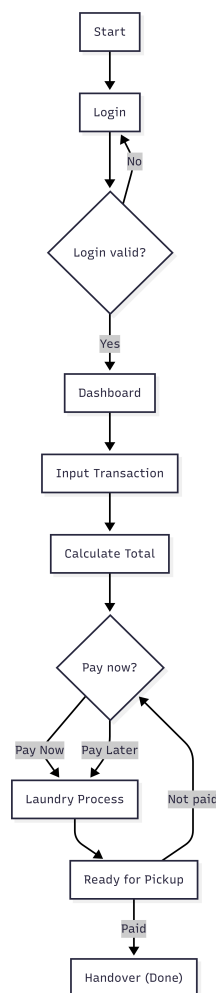
View ini berfungsi untuk mentransformasi data waktu transaksi secara real-time ke format Waktu Indonesia Barat (WIB), sehingga memudahkan proses pelaporan

tanpa mengubah data asli pada tabel utama.

3.2.4 Algoritma fungsi

Dalam pengembangan sistem point-of-sale laundry berbasis web ini, algoritma fungsional digunakan untuk menentukan logika bisnis utama, seperti total biaya berdasarkan berat atau kuantitas, pemantauan status cucian secara real-time, dan laporan keuangan otomatis. Algoritma ini digunakan untuk memastikan proses kerja sistematis, akurat, dan meminimalkan kesulitan yang terkait dengan pekerjaan manual.

3.2.5 Flowchart



Gambar III.4: Flowchart

Flowchart pada Gambar diatas merepresentasikan algoritma utama atau logika bisnis yang berjalan di dalam Sistem POS WellClean Laundry. Alur ini menggam-

barkan urutan eksekusi proses dari awal akses hingga penyelesaian layanan (handover). Berikut adalah rincian tahapan prosesnya:

1. Otentikasi Pengguna (Login): Proses dimulai dengan tahapan login. Sistem memvalidasi kredensial pengguna melalui percabangan keputusan (decision node) "Login valid?". Jika data tidak valid, pengguna akan dikembalikan ke halaman login. Jika valid, pengguna diberikan akses menuju Dashboard utama.
2. Input Transaksi dan Kalkulasi: Dari Dashboard, pengguna melakukan Input Transaction (memasukkan data pelanggan dan layanan). Sistem kemudian memproses data tersebut pada tahap Calculate Total untuk menghasilkan total biaya yang harus dibayarkan secara otomatis.
3. Fleksibilitas Pembayaran (Pay Now vs Pay Later): Sistem dirancang untuk menangani dua skenario pembayaran melalui percabangan "Pay now?":
 - Pay Now: Pelanggan membayar lunas di awal transaksi.
 - Pay Later: Pelanggan memilih untuk membayar nanti (bayar di akhir). Kedua pilihan ini tetap akan meneruskan alur ke tahapan produksi atau Laundry Process.
4. Proses Produksi dan Pengambilan: Setelah proses pencucian selesai (Laundry Process), status cucian diperbarui menjadi Ready for Pickup (Siap Diambil). Pada tahap ini, terdapat mekanisme validasi akhir sebelum barang diserahkan:
 - Jika status pembayaran masih Not Paid (belum lunas), alur akan dikembalikan (looping) ke menu pembayaran untuk menyelesaikan administrasi.
 - Jika status sudah Paid (lunas), maka sistem mengizinkan proses Handover (Serah Terima) dan transaksi dinyatakan selesai (Done).

BAB IV

IMPLEMENTASI DAN PENGUJIAN

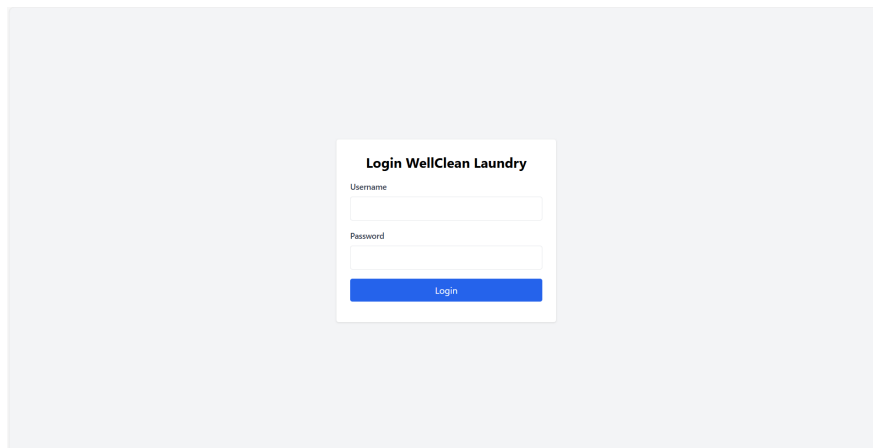
4.1 Pembahasan Hasil Implementasi

Tahap implementasi merupakan fase di mana rancangan sistem yang telah dibuat sebelumnya direalisasikan ke dalam bentuk kode program menggunakan bahasa pemrograman Go (Golang) dan antarmuka berbasis web. Pada bagian ini, akan dijelaskan dua aspek utama implementasi, yaitu implementasi antarmuka pengguna (User Interface) untuk memberikan gambaran visual sistem, serta implementasi logika program (backend) yang menjelaskan alur kerja setiap fungsi secara mendetail.

4.1.1 Implementasi Antarmuka Pengguna (User Interface)

Implementasi antarmuka pada Sistem POS Laundry ini dirancang dengan prinsip usability untuk memastikan pengguna, baik admin maupun karyawan, dapat mengoperasikan sistem dengan mudah. Setiap halaman dirancang secara responsif guna mendukung fleksibilitas akses dari berbagai perangkat.

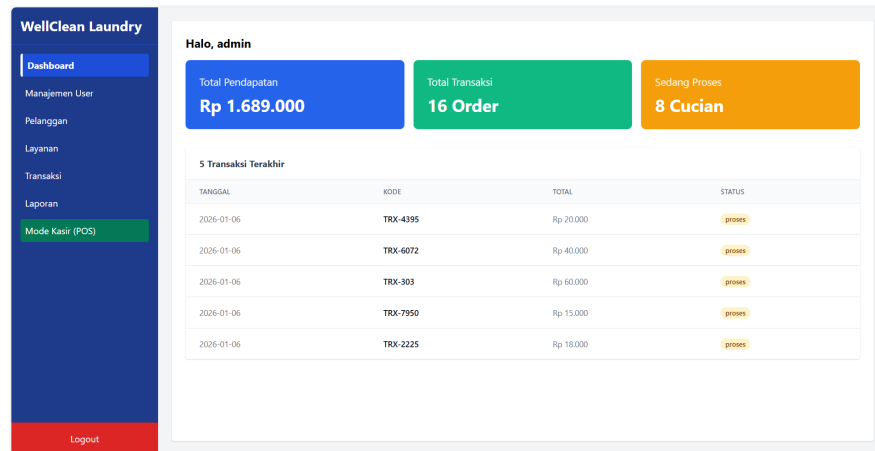
1. Halaman Otentikasi (Login)



Gambar IV.5: Halaman Otentikasi (Login)

Halaman ini merupakan pintu masuk utama sekaligus garda depan keamanan sistem. Secara visual, halaman ini menyediakan form input untuk kredensial pengguna. Secara sistem, halaman ini berfungsi untuk memvalidasi identitas pengguna melalui pencocokan data yang ada pada database dengan teknik enkripsi tertentu guna menjaga kerahasiaan informasi.

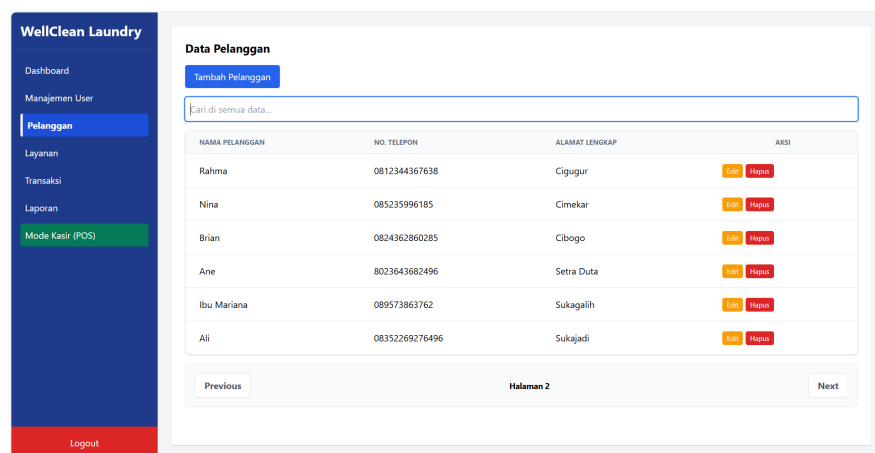
2. Dashboard Operasional Utama



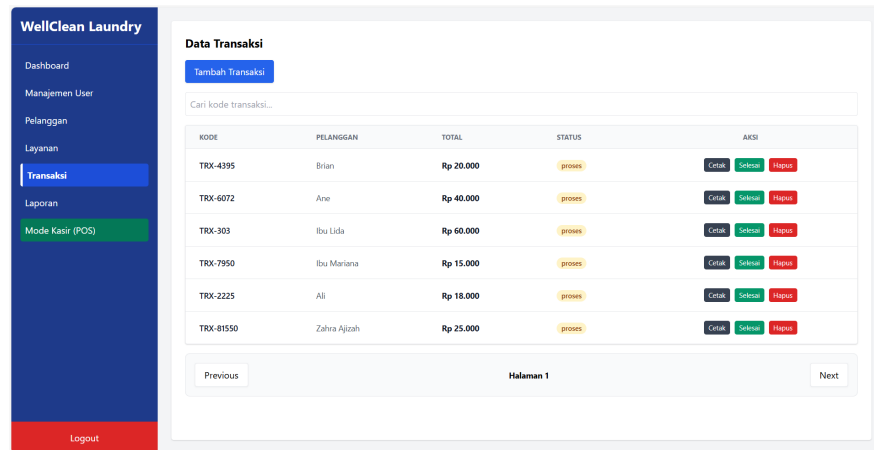
Gambar IV.6: Halaman Dashboard

Dashboard ini merupakan pusat informasi dari seluruh aktivitas laundry. Implementasi halaman ini mencakup penyajian data transaksi harian, status pengerjaan cucian (Proses dan Selesai), total orderan, hingga total jumlah pendapatan. Informasi yang disajikan bersifat aktual (real-time) sehingga pemilik usaha dapat melakukan pemantauan operasional secara efisien tanpa harus melakukan pembaruan halaman secara manual.

3. Manajemen Pelanggan dan Transaksi



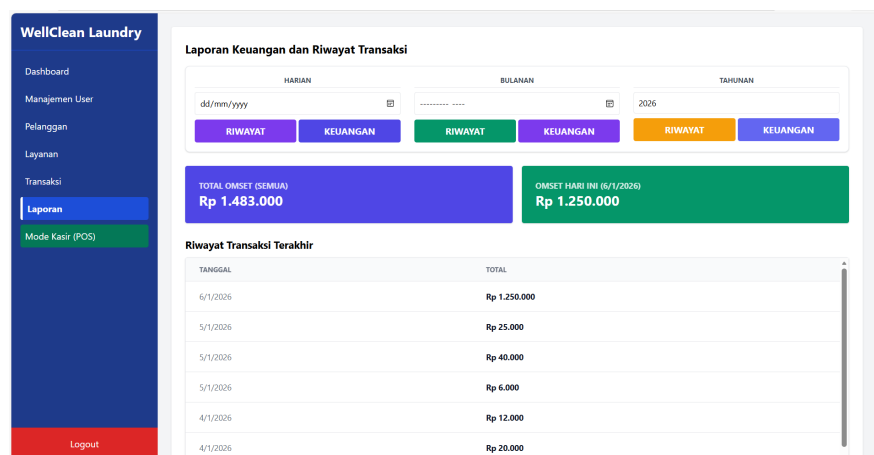
Gambar IV.7: Halaman Manajemen Pelanggan



Gambar IV.8: Halaman Manajemen Transaksi

Halaman ini mengimplementasikan fungsi CRUD (Create, Read, Update, Delete) untuk mengelola data pelanggan dan riwayat transaksi. Antarmuka pada bagian ini fokus pada akurasi input data untuk mengurangi risiko kesalahan pencatatan manual yang sering terjadi pada sistem konvensional.

4. Halaman Laporan Keuangan dan Riwayat Transaksi



Gambar IV.9: Halaman Laporan

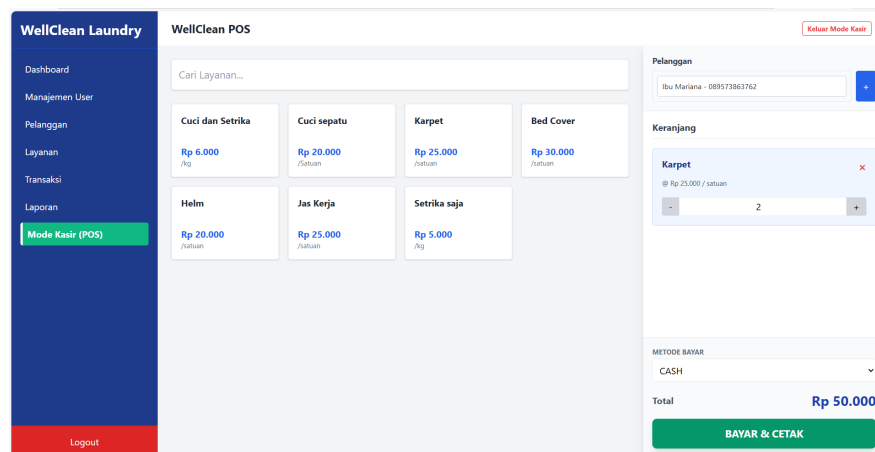
Halaman ini berfungsi sebagai pusat dokumentasi finansial usaha laundry. Antarmuka laporan dirancang untuk memberikan fleksibilitas kepada pemilik dalam memantau laporan omset secara harian, bulanan, hingga tahunan. Dalam Visual dan Fungsi:

- Visual: Terdapat kartu ringkasan (summary cards) yang menampilkan total omset keseluruhan dan omset spesifik pada hari berjalan (seperti

contoh pada Gambar 4.1.5 yang menunjukkan Omset Hari Ini sebesar Rp 1.250.000).

- Fungsi: Implementasi pada halaman ini memungkinkan pengguna untuk melihat tabel riwayat transaksi terakhir secara mendetail. Selain itu, terdapat tombol filtrasi laporan yang terhubung langsung dengan modul pengunduhan data (Export Excel).

5. Halaman Mode Kasir (Point of Sale)



Gambar IV.10: Halaman Kasir

Mode Kasir (POS) merupakan fitur utama yang digunakan untuk melayani pelanggan secara langsung. Antarmuka ini dirancang dengan mengutamakan kecepatan proses input agar pelayanan tidak terhambat.

- Fungsi: Pada halaman ini, karyawan dapat memilih jenis layanan (laundry kiloan/satuan), memasukkan data pelanggan, dan menentukan metode pembayaran.
- Output: Setiap transaksi yang diselesaikan melalui Mode Kasir akan secara otomatis memperbarui saldo omset di halaman laporan dan memicu pengiriman data melalui sistem SSE (Server-Sent Events) ke dashboard utama.

4.1.2 Implementasi Logika Program (Bedah Kode)

Pada bagian ini akan dijelaskan alur logika dari fungsi-fungsi utama yang menjalankan Sistem POS Laundry. Penjelasan difokuskan pada bagaimana setiap baris kode menangani data masuk (input), melakukan pengolahan (proses), hingga menghasilkan informasi yang dibutuhkan (output).

1. Modul Webhook Pembayaran (webhook.go)

Berikut adalah potongan kode untuk fungsi GopayHandler dan StreamHandler:

```
package main

import (
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "regexp"
    "strings"
)

var pesanChan = make(chan string, 10)

func GopayHandler(w http.ResponseWriter, r *http.Request)

{
    bodyBytes, _ := io.ReadAll(r.Body)

    var incoming struct {
        NotificationText string `json:"notification_text"`
    }
    json.Unmarshal(bodyBytes, &incoming)

    re := regexp.MustCompile(`(\d[\d\.]+)`)
    match := re.FindString(incoming.NotificationText)

    finalText := incoming.NotificationText
    if match != "" {
        angkaBersih := strings.ReplaceAll(match, ".", "")
        finalText = "Pembayaran Rp " + angkaBersih + "

        Berhasil!"
    }

    pesanChan <- finalText
```

```

        fmt.Println("Berhasil kirim ke dashboard:", finalText)
        w.WriteHeader(http.StatusOK)
    }

func StreamHandler(w http.ResponseWriter, r *http.Request)

{
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")
    w.Header().Set("Access-Control-Allow-Origin", "*")

    notify := r.Context().Done()

    for {
        select {
        case <-notify:
            return
        case pesan := <-pesanChan:
            fmt.Fprintf(w, "data: %s\n\n", pesan)

            if f, ok := w.(http.Flusher); ok {
                f.Flush()
            }
        }
    }
}

```

Penjelasan kode:

(a) Input: Fungsi menerima objek `r *http.Request` yang berisi paket data JSON dengan kunci `notification`, *ext dari payment gateway*.

(a) Proses:

- i. Baris `io.ReadAll(r.Body)` membaca seluruh aliran data mentah yang masuk.
- ii. `json.Unmarshal` mengubah data mentah tersebut ke dalam struktur

variabel agar teks notifikasi bisa diolah.

iii. `regexp.MustCompile` dan `re.FindString` bekerja sama mencari pola angka di dalam teks untuk mendeteksi nominal pembayaran secara otomatis.

iv. `strings.ReplaceAll` digunakan untuk membersihkan karakter titik agar nominal menjadi angka bulat yang siap disimpan.

(b) Output: Mengirimkan variabel `finalText` ke dalam `pesanChan` untuk diteruskan ke dashboard dan memberikan respon HTTP 200 OK ke server pengirim.

2. Modul Utama dan Manajemen Akun (`main.go`) Pada file `main.go`, seluruh logika bisnis dan integrasi database Supabase dikelola. Berikut adalah bedah baris kode berdasarkan fungsinya:

```
package main

import (
    "bytes"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "net/url"
    "strconv"
    "strings"
    "time"

    "golang.org/x/crypto/bcrypt"

    "github.com/golang-jwt/jwt/v5"
    "github.com/xuri/excelize/v2"
)

type User struct {
    ID          string `json:"id"`
    Username    string `json:"username"`
    Password    string `json:"password"`
    Role        string `json:"role"`
}
```

```

type LoginRequest struct {
    Username string `json:"username"`
    Password string `json:"password"`
}

func enableCORS(w http.ResponseWriter) {
    w.Header().Set("Access-Control-Allow-Origin", "*")
    w.Header().Set("Access-Control-Allow-Headers",

        "Content-Type, Authorization")
    w.Header().Set("Access-Control-Allow-Methods",

        "GET, POST, PATCH, DELETE, OPTIONS")
}

func corsMiddleware(next http.HandlerFunc)

http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        enableCORS(w)
        if r.Method == http.MethodOptions {
            w.WriteHeader(http.StatusOK)
            return
        }
        next(w, r)
    }
}

func validateToken(r *http.Request) (jwt.MapClaims,

error) {
    authHeader := r.Header.Get("Authorization")
    if authHeader == "" {
        return nil, fmt.Errorf("missing token")
    }
}

```

```

tokenString := strings.TrimPrefix(authHeader,

"Bearer ")
token, err := jwt.Parse(tokenString, func(token *jwt.

Token) (interface{}, error) {
    if _, ok := token.Method.(*jwt.SigningMethodHMAC);

        !ok {
            return nil, fmt.Errorf("unexpected signing

                method: %v", token.Header["alg"])
        }
        return []byte(JWTSecret), nil
    })

    if err != nil {
        return nil, err
    }

    if claims, ok := token.Claims.(jwt.MapClaims); ok &&

        token.Valid {
            return claims, nil
        }

        return nil, fmt.Errorf("invalid token")
    }

func loginHandler(w http.ResponseWriter, r *http.Request)

{
    if r.Method != http.MethodPost {

```



```

        w.WriteHeader(http.StatusMethodNotAllowed)
        return
    }

    var req LoginRequest
    if err := json.NewDecoder(r.Body).Decode(&req);

    err != nil {
        w.WriteHeader(http.StatusBadRequest)
        json.NewEncoder(w).Encode(map[string]string{

            "error": "invalid body"})
        return
    }

    // 1. Cari user di tabel public.users
    client := &http.Client{}
    url := fmt.Sprintf("%s/rest/v1/users?username=req.

    %s&select=*", BaseURL, req.Username)
    reqSup, _ := http.NewRequest("GET", url, nil)
    reqSup.Header.Set("apikey", APIKey)
    reqSup.Header.Set("Authorization", "Bearer "+APIKey)

    resSup, err := client.Do(reqSup)
    if err != nil {
        w.WriteHeader(http.StatusInternalServerError)
        json.NewEncoder(w).Encode(map[string]string{

            "error": "Supabase Connection Error: " + err.Error()})
        return
    }
    defer resSup.Body.Close()

    if resSup.StatusCode >= 400 {
        var supError map[string]interface{}

```

```

    json.NewDecoder(resSup.Body).Decode(&supError)
    w.WriteHeader(resSup.StatusCode)
    json.NewEncoder(w).Encode(map[string]any{

        "error": "Supabase Error", "details": supError})
    return
}

var users []User
if err := json.NewDecoder(resSup.Body).Decode(&users);

err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    json.NewEncoder(w).Encode(map[string]string{

        "error": "Failed to decode users: " + err.Error()})
    return
}

if len(users) == 0 {
    w.WriteHeader(http.StatusUnauthorized)
    json.NewEncoder(w).Encode(map[string]string{

        "error": "User tidak ditemukan"})
    return
}

user := users[0]

if !CheckPasswordHash(req.Password, user.Password) {
    w.WriteHeader(http.StatusUnauthorized)
    json.NewEncoder(w).Encode(map[string]string{

        "error": "Password salah",
    })
    return
}

```

```

// 3. Buat JWT
token := jwt.NewWithClaims(jwt.SigningMethodHS256,

jwt.MapClaims{
    "sub":      user.ID,
    "username": user.Username,
    "role":     user.Role,
    "exp":      time.Now().Add(time.Hour * 24).Unix(),
})

tokenString, err := token.SignedString([]byte

(JWTSecret))
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(map[string]any{
    "access_token": tokenString,
    "role":         user.Role,
})
}

func registerHandler(w http.ResponseWriter, r

*http.Request) {

    claims, err := validateToken(r)
    if err != nil {
        fmt.Println("Gagal Verifikasi Karena:", err)

        w.WriteHeader(http.StatusUnauthorized)
        json.NewEncoder(w).Encode(map[string]string{

```

```

        "error": err.Error()})
    return
}

if claims["role"] != "admin" {
    w.WriteHeader(http.StatusForbidden)
    json.NewEncoder(w).Encode(map[string]string{

        "error": "Hanya admin yang bisa menambah kasir"})
    return
}

var input struct {
    Username string `json:"username"`
    Password string `json:"password"`
    Nama      string `json:"nama"`
}

if err := json.NewDecoder(r.Body).Decode(&input);

err != nil {
    w.WriteHeader(http.StatusBadRequest)
    return
}

hashed, err := HashPassword(input.Password)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}

userData := map[string]any{
    "username": input.Username,
    "password": hashed,
    "nama":     input.Nama,
    "role":     "kasir",
}

```

```

body, _ := json.Marshal(userData)

client := &http.Client{}
url := BaseURL + "/rest/v1/users"
reqSup, _ := http.NewRequest("POST", url, bytes.

NewBuffer(body))
reqSup.Header.Set("apikey", APIKey)
reqSup.Header.Set("Authorization", "Bearer "+APIKey)
reqSup.Header.Set("Content-Type", "application/json")

resSup, err := client.Do(reqSup)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
defer resSup.Body.Close()

if resSup.StatusCode >= 400 {
    w.WriteHeader(resSup.StatusCode)
    return
}

w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated)
json.NewEncoder(w).Encode(map[string]string{"message":

    "Kasir berhasil didaftarkan"})
}

func getUsersHandler(w http.ResponseWriter, r

*http.Request) {
    claims, err := validateToken(r)
    if err != nil || claims["role"] != "admin" {
        w.WriteHeader(http.StatusForbidden)

```

```

        json.NewEncoder(w).Encode(map[string]string{

            "error": "Forbidden"})
        return
    }

    client := &http.Client{}
    url := BaseURL + "/rest/v1/users?select=*"

    reqSup, _ := http.NewRequest("GET", url, nil)
    reqSup.Header.Set("apikey", APIKey)
    reqSup.Header.Set("Authorization", "Bearer "+APIKey)

    resSup, err := client.Do(reqSup)
    if err != nil {
        w.WriteHeader(http.StatusInternalServerError)
        return
    }
    defer resSup.Body.Close()

    body, _ := io.ReadAll(resSup.Body)

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(resSup.StatusCode)
    w.Write(body)
}

func deleteUserHandler(w http.ResponseWriter, r

*http.Request) {
    claims, err := validateToken(r)
    if err != nil || claims["role"] != "admin" {
        w.WriteHeader(http.StatusUnauthorized)
        return
    }

    id := r.URL.Query().Get("id")
    if id == "" {

```

```

        w.WriteHeader(http.StatusBadRequest)
        return
    }

    client := &http.Client{}
    reqSup, _ := http.NewRequest("DELETE",

    BaseURL+"/rest/v1/users?id=eq."+id, nil)
    reqSup.Header.Set("apikey", APIKey)
    reqSup.Header.Set("Authorization", "Bearer "+APIKey)

    resSup, _ := client.Do(reqSup)
    w.WriteHeader(resSup.StatusCode)
}

func updateUserHandler(w http.ResponseWriter, r
*http.Request) {
    claims, err := validateToken(r)
    if err != nil || claims["role"] != "admin" {
        w.WriteHeader(http.StatusUnauthorized)
        return
    }

    id := r.URL.Query().Get("id")
    var input map[string]interface{}
    json.NewDecoder(r.Body).Decode(&input)

    body, _ := json.Marshal(input)
    client := &http.Client{}
    reqSup, _ := http.NewRequest("PATCH",

    BaseURL+"/rest/v1/users?id=eq."+id,

    bytes.NewBuffer(body))
    reqSup.Header.Set("apikey", APIKey)

```

```

reqSup.Header.Set("Authorization", "Bearer "+APIKey)
reqSup.Header.Set("Content-Type", "application/json")

resSup, _ := client.Do(reqSup)
w.WriteHeader(resSup.StatusCode)
}

func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte

        (password), bcrypt.DefaultCost)
    return string(bytes), err
}

func CheckPasswordHash(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash),

        []byte(password))
    return err == nil
}

func dashboardHandler(w http.ResponseWriter, r

*http.Request) {
    claims, err := validateToken(r)
    if err != nil {
        w.WriteHeader(http.StatusUnauthorized)
        json.NewEncoder(w).Encode(map[string]string{

            "error": err.Error()})
        return
    }

    client := &http.Client{}
    // Pakai Key Anon untuk ambil data transaksi
    transaksiURL := BaseURL + "/rest/v1/transaksi?"

```



```

select=*&order=created_at.desc"
reqTrans, _ := http.NewRequest("GET", transaksiURL,

nil)
reqTrans.Header.Set("apikey", APIKey)
reqTrans.Header.Set("Authorization", "Bearer "+APIKey)

resTrans, err := client.Do(reqTrans)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
defer resTrans.Body.Close()

var transaksiList []map[string]any
if resTrans.StatusCode == http.StatusOK {
    json.NewDecoder(resTrans.Body).Decode

    (&transaksiList)
} else {
    transaksiList = []map[string]any{}
}

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(map[string]any{
    "ok":      true,
    "user":    claims["username"],
    "role":    claims["role"],
    "transaksi": transaksiList,
})
}

func pelangganHandler(w http.ResponseWriter, r
*http.Request) {

```

```

_, err := validateToken(r)
if err != nil {
    w.WriteHeader(http.StatusUnauthorized)
    json.NewEncoder(w).Encode(map[string]string{

        "error": err.Error()})
    return
}

query := r.URL.Query()
searchTerm := query.Get("search")

bodyBytes, _ := io.ReadAll(r.Body)
r.Body = io.NopCloser(bytes.NewBuffer(bodyBytes))

filterQuery := ""
if searchTerm != "" {
    filterQuery = fmt.Sprintf("&or=(nama.ilike.*%s*,

        telepon.ilike.*%s*)", searchTerm, searchTerm)
}

fullURL := BaseURL + "/rest/v1/pelanggan?select=*"
if r.Method == http.MethodGet {
    fullURL += filterQuery
}

page, _ := strconv.Atoi(query.Get("page"))
limit, _ := strconv.Atoi(query.Get("limit"))

query.Del("page")
query.Del("limit")
query.Del("search")

if len(query) > 0 {
    fullURL += "&" + query.Encode()
}

```

```

reqSup, _ := http.NewRequest(r.Method, fullURL,

bytes.NewBuffer(bodyBytes))

reqSup.Header.Set("Content-Type", "application/json")
reqSup.Header.Set("apikey", APIKey)
reqSup.Header.Set("Authorization", "Bearer "+APIKey)

if r.Method == http.MethodGet && page > 0 {
    if limit < 1 {
        limit = 10
    }
    from := (page - 1) * limit
    to := from + limit - 1
    reqSup.Header.Set("Range", fmt.Sprintf("%d-%d",

        from, to))
}

client := &http.Client{}
resSup, err := client.Do(reqSup)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
defer resSup.Body.Close()

resBody, _ := io.ReadAll(resSup.Body)
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(resSup.StatusCode)
w.Write(resBody)
}

func transaksiHandler(w http.ResponseWriter, r

*http.Request) {
    _, err := validateToken(r)

```

```

if err != nil {
    w.WriteHeader(http.StatusUnauthorized)
    json.NewEncoder(w).Encode(map[string]string{

        "error": err.Error()})
    return
}

query := r.URL.Query()
page, _ := strconv.Atoi(query.Get("page"))
limit, _ := strconv.Atoi(query.Get("limit"))
searchTerm := query.Get("search")

bodyBytes, _ := io.ReadAll(r.Body)
r.Body = io.NopCloser(bytes.NewBuffer(bodyBytes))

fullURL := BaseURL + "/rest/v1/transaksi"
params := url.Values{}
params.Add("select", "*")
params.Add("order", "created_at.desc")

if r.Method == http.MethodGet && searchTerm != "" {
    params.Add("kode", "ilike.*"+searchTerm+"*")
}

query.Del("page")
query.Del("limit")
query.Del("search")
query.Del("order")
for k, v := range query {
    params.Add(k, v[0])
}
fullURL += "?" + params.Encode()

client := &http.Client{}
reqSup, _ := http.NewRequest(r.Method, fullURL,

bytes.NewBuffer(bodyBytes))

```

```

reqSup.Header.Set("Content-Type", "application/json")
reqSup.Header.Set("apikey", APIKey)
reqSup.Header.Set("Authorization", "Bearer "+APIKey)

if r.Method == http.MethodGet && page > 0 {
    if limit < 1 {
        limit = 10
    }
    from := (page - 1) * limit
    to := from + limit - 1
    reqSup.Header.Set("Range", fmt.Sprintf("%d-%d",

        from, to))
}

resSup, err := client.Do(reqSup)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
defer resSup.Body.Close()

resBody, _ := io.ReadAll(resSup.Body)
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(resSup.StatusCode)
w.Write(resBody)
}

func layananHandler(w http.ResponseWriter, r

    *http.Request) {
    claims, err := validateToken(r)
    if err != nil {
        w.WriteHeader(http.StatusUnauthorized)
        json.NewEncoder(w).Encode(map[string]string{

            "error": err.Error() })
    }
}

```

```

        return
    }

    method := r.Method
    if method == http.MethodPost || method ==

http.MethodDelete ||

method == http.MethodPatch || method ==

http.MethodPut {
    role, _ := claims["role"].(string)
    if role != "admin" {
        w.WriteHeader(http.StatusForbidden)
        json.NewEncoder(w).Encode(map[string]string{

            "error": "Akses ditolak: Hanya admin"})
        return
    }
}

query := r.URL.Query()
page, _ := strconv.Atoi(query.Get("page"))
limit, _ := strconv.Atoi(query.Get("limit"))
searchTerm := query.Get("search")

bodyBytes, _ := io.ReadAll(r.Body)
r.Body = io.NopCloser(bytes.NewBuffer(bodyBytes))

fullURL := BaseURL + "/rest/v1/layanan"
params := url.Values{}
params.Add("select", "*")

if method == http.MethodGet && searchTerm != "" {
    params.Add("nama", "ilike.*"+searchTerm+"*")
}

```

```

query.Del("page")
query.Del("limit")
query.Del("search")
for k, v := range query {
    params.Add(k, v[0])
}

fullURL += "?" + params.Encode()

client := &http.Client{}
reqSup, _ := http.NewRequest(method, fullURL,

bytes.NewBuffer(bodyBytes))
reqSup.Header.Set("Content-Type", "application/json")
reqSup.Header.Set("apikey", APIKey)
reqSup.Header.Set("Authorization", "Bearer "+APIKey)

if method == http.MethodGet && page > 0 {
    if limit < 1 {
        limit = 10
    }
    from := (page - 1) * limit
    to := from + limit - 1
    reqSup.Header.Set("Range", fmt.Sprintf("%d-%d",

        from, to))
}

resSup, err := client.Do(reqSup)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
defer resSup.Body.Close()

resBody, _ := io.ReadAll(resSup.Body)
w.Header().Set("Content-Type", "application/json")

```

```

        w.WriteHeader(resSup.StatusCode)
        w.Write(resBody)
    }

type Transaksi struct {
    Kode            string    `json:"kode"`
    CreatedAt       time.Time `json:"created_at"`
    SelesaiAt       *time.Time `json:"selesai_at"`
    PelangganNama   string    `json:"pelanggan_nama"`
    LayananNama     string    `json:"layanan_nama"`
    Berat           float64   `json:"berat"`
    HargaSatuan     float64   `json:"harga_satuan"`
    Total           float64   `json:"total"`
    MetodePembayaran string    `json:"metode_pembayaran"`
    Status          string    `json:"status"`
}

func laporanHandler(w http.ResponseWriter, r

*http.Request) {
    _, err := validateToken(r)
    if err != nil {
        w.WriteHeader(http.StatusUnauthorized)
        return
    }

    tipe := r.URL.Query().Get("type")
    client := &http.Client{}

    if tipe == "riwayat" {
        transaksiURL := BaseURL +
            "/rest/v1/transaksi_wib" +
            "?status=eq.selesai" +
            "&select=kode,created_at,selesai_at,berat,

            total,metode_pembayaran,status,"
    }
}

```



```

    pelanggan(nama),

    layanan(nama) " +
    "&order=selesai_at.desc"

req, _ := http.NewRequest("GET", transaksiURL,

nil)
req.Header.Set("apikey", APIKey)
req.Header.Set("Authorization", "Bearer "+APIKey)

res, err := client.Do(req)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
defer res.Body.Close()

var raw []map[string]any
json.NewDecoder(res.Body).Decode(&raw)

trx := []Transaksi{}
for _, item := range raw {
    t := Transaksi{}

    t.Kode = fmt.Sprint(item["kode"])
    t.Status = fmt.Sprint(item["status"])
    t.MetodePembayaran = fmt.Sprint(item[

"metode_pembayaran"])

    if p, ok := item["pelanggan"].(map[string]any)

; ok {
        t.PelangganNama = fmt.Sprint(p["nama"])
    }
}

```

```

        if l, ok := item["layanana"].(map[string]any)

        ; ok {
            t.LayananNama = fmt.Sprintf(l["nama"])
        }

        if s, ok := item["selesai_at"].(string); ok

        && s != "" {
            if tm, err := time.Parse(time.RFC3339, s);

            err == nil {
                t.SelesaiAt = &tm
            }
        }

        if total, ok := item["total"].(float64); ok {
            t.Total = total
        }

        trx = append(trx, t)
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]any{"data":

    trx})
    return
}

if tipe == "keuangan" {
    loc, _ := time.LoadLocation("Asia/Jakarta")
    today := time.Now().In(loc).Format("2006-01-02")

    getSum := func(url string) float64 {
        req, _ := http.NewRequest("GET", url, nil)

```

```

req.Header.Set("apikey", APIKey)
req.Header.Set("Authorization", "Bearer

"+APIKey)

res, err := client.Do(req)
if err != nil {
    return 0
}
defer res.Body.Close()

var list []map[string]any
json.NewDecoder(res.Body).Decode(&list)

var sum float64
for _, i := range list {
    if t, ok := i["total"].(float64); ok {
        sum += t
    }
}
return sum
}

// TOTAL OMSET (SEMUA)
totalURL := BaseURL +
    "/rest/v1/transaksi_wib" +
    "?status=eq.selesai&select=total"

// OMSET HARIAN (WIB FIX)
dailyURL := BaseURL +
    "/rest/v1/transaksi_wib" +
    "?status=eq.selesai" +
    "&selesai_tanggal_wib=eq." + today +
    "&select=total"

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(map[string]any{
    "total_omset": getSum(totalURL),
    "omset_harian": getSum(dailyURL),

```

```

        })
        return
    }

    w.WriteHeader(http.StatusBadRequest)
}

func laporanExportHandler(w http.ResponseWriter, r
*http.Request) {
    claims, err := validateToken(r)
    if err != nil {
        w.WriteHeader(http.StatusUnauthorized)
        return
    }

    role, _ := claims["role"].(string)
    if role != "admin" {
        w.WriteHeader(http.StatusForbidden)
        return
    }

    tipe := r.URL.Query().Get("type")
    periode := r.URL.Query().Get("periode")

    loc, _ := time.LoadLocation("Asia/Jakarta")
    var start, end time.Time

    switch periode {
    case "harian":
        valDate := r.URL.Query().Get("date")
        tgl, _ := time.ParseInLocation("2006-01-02",

        valDate, loc)
        start = time.Date(tgl.Year(), tgl.Month(),

        tgl.Day(), 0, 0, 0, 0, loc)

```

```

        end = time.Date(tgl.Year(), tgl.Month(),

        tgl.Day(), 23, 59, 59, 0, loc)

    case "bulanan":
        y, _ := strconv.Atoi(r.URL.Query().Get("year"))
        m, _ := strconv.Atoi(r.URL.Query().Get("month"))
        start = time.Date(y, time.Month(m), 1, 0, 0, 0, 0,
        loc)
        end = start.AddDate(0, 1, -1)
        end = time.Date(end.Year(), end.Month(), end.Day(),

        23, 59, 59, 0, loc)

    case "tahunan":
        y, _ := strconv.Atoi(r.URL.Query().Get("year"))
        start = time.Date(y, 1, 1, 0, 0, 0, 0, loc)
        end = time.Date(y, 12, 31, 23, 59, 59, 0, loc)

    default:
        now := time.Now().In(loc)
        start = time.Date(now.Year(), now.Month(),

        now.Day(), 0, 0, 0, 0, loc)
        end = start.AddDate(0, 0, 1)
    }

    filter := fmt.Sprintf("selesai_tanggal_wib=gte.%s&

    selesai_tanggal_wib=lte.%s",
        start.Format("2006-01-02"),
        end.Format("2006-01-02"))

    var transaksiURL string

```

```

if tipe == "riwayat" {
    transaksiURL = BaseURL + "/rest/v1/transaksi_wib?

    status=eq.selesai&select=kode,

    selesai_tanggal_wib,berat,total,

    metode_pembayaran,status,pelanggan(nama),

    layanan(nama)"
} else {
    transaksiURL = BaseURL + "/rest/v1/transaksi_wib?

    status=eq.selesai&select=selesai_tanggal_wib,

    total"
}

transaksiURL += "&" + filter

client := &http.Client{}
req, _ := http.NewRequest("GET", transaksiURL, nil)
req.Header.Set("apikey", APIKey)
req.Header.Set("Authorization", "Bearer "+APIKey)

res, err := client.Do(req)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
defer res.Body.Close()

var raw []map[string]any
json.NewDecoder(res.Body).Decode(&raw)

```

```

var trx []Transaksi
var totalOmsetPeriode float64

for _, item := range raw {
    t := Transaksi{}
    t.Kode = fmt.Sprint(item["kode"])
    t.Status = fmt.Sprint(item["status"])
    t.MetodePembayaran = fmt.Sprint(item[

        "metode_pembayaran"])

    if p, ok := item["pelanggan"].(map[string]any); ok {

        {
            t.PelangganNama = fmt.Sprint(p["nama"])
        }
        if l, ok := item["layanan"].(map[string]any); ok {
            t.LayananNama = fmt.Sprint(l["nama"])
        }

        if s, ok := item["selesai_tanggal_wib"].(string);

        ok && s != "" {
            if tm, err := time.Parse(time.RFC3339, s); err

                == nil {
                    t.SelesaiAt = &tm
                } else if tm, err := time.Parse("2006-01-02",

                    s); err == nil {
                        t.SelesaiAt = &tm
                    }
                }
            }
        }
    }
}

```

```

    if total, ok := item["total"].(float64); ok {
        t.Total = total
    } else if totalStr, ok := item["total"].

(string); ok {
        t.Total, _ = strconv.ParseFloat(totalStr, 64)
    }

    if berat, ok := item["berat"].(float64); ok {
        t.Berat = berat
    }
    if t.Berat > 0 {
        t.HargaSatuan = t.Total / t.Berat
    }

    totalOmsetPeriode += t.Total
    trx = append(trx, t)
}

f := excelize.NewFile()
sheet := "Laporan"
f.SetSheetName("Sheet1", sheet)

headerStyle, _ := f.NewStyle(&excelize.Style{
    Font:      &excelize.Font{Bold: true, Color:

"FFFFFF"},
    Fill:      excelize.Fill{Type: "pattern", Color:

[]string{"4B0082"}, Pattern: 1},
    Alignment: &excelize.Alignment{Horizontal:

"center"},
})
totalStyle, _ := f.NewStyle(&excelize.Style{
    Font: &excelize.Font{Bold: true},

```



```

Fill: excelize.Fill{Type: "pattern", Color:

[]string{"E0E0E0"}, Pattern: 1},
}))

if tipe == "riwayat" {
    headers := []string{"Kode", "Tanggal Selesai",

"Pelanggan", "Layanan", "Berat", "Harga Satuan",

"Total", "Metode", "Status"}
    for i, h := range headers {
        col := string(rune('A' + i))
        f.SetCellValue(sheet, col+"1", h)
        f.SetCellStyle(sheet, col+"1", col+"1",

headerStyle)
        f.SetColWidth(sheet, col, col, 18)
    }

    for i, t := range trx {
        row := strconv.Itoa(i + 2)
        f.SetCellValue(sheet, "A"+row, t.Kode)
        if t.SelesaiAt != nil {
            f.SetCellValue(sheet, "B"+row,

t.SelesaiAt.In(loc).Format("02-01-2006"))
        }
        f.SetCellValue(sheet, "C"+row,

t.PelangganNama)
        f.SetCellValue(sheet, "D"+row, t.LayananNama)
        f.SetCellValue(sheet, "E"+row, t.Berat)
        f.SetCellValue(sheet, "F"+row, t.HargaSatuan)
    }
}

```

```

        f.SetCellValue(sheet, "G"+row, t.Total)
        f.SetCellValue(sheet, "H"+row,

        t.MetodePembayaran)
        f.SetCellValue(sheet, "I"+row, t.Status)
    }

    lastRow := strconv.Itoa(len(trx) + 2)
    f.SetCellValue(sheet, "F"+lastRow, "TOTAL")
    f.SetCellValue(sheet, "G"+lastRow,

    totalOmsetPeriode)
    f.SetCellStyle(sheet, "F"+lastRow, "G"+lastRow,

    totalStyle)

} else {
    f.SetCellValue(sheet, "A1", "Tanggal Selesai")
    f.SetCellValue(sheet, "B1", "Omset (Rp)")
    f.SetCellStyle(sheet, "A1", "B1", headerStyle)
    f.SetColWidth(sheet, "A", "B", 25)

    for i, t := range trx {
        row := strconv.Itoa(i + 2)
        if t.SelesaiAt != nil {
            f.SetCellValue(sheet, "A"+row,

            t.SelesaiAt.In(loc).Format("02-01-2006"))
        }
        f.SetCellValue(sheet, "B"+row, t.Total)
    }

    lastRow := strconv.Itoa(len(trx) + 2)
    f.SetCellValue(sheet, "A"+lastRow,

```

```

        "TOTAL OMSET PERIODE INI")
        f.SetCellValue(sheet, "B"+lastRow,

        totalOmsetPeriode)
        f.SetCellStyle(sheet, "A"+lastRow,

        "B"+lastRow, totalStyle)
    }

    var buf bytes.Buffer
    f.Write(&buf)

    w.Header().Set("Content-Type",

    "application/vnd.openxmlformats-officedocument.

    spreadsheetml.sheet")
    w.Header().Set("Content-Disposition",

    fmt.Sprintf("attachment; filename=Laporan_%s_%s.xlsx",

    tipe, start.Format("2006-01-02")))
    w.Write(buf.Bytes())
}

func main() {
    http.HandleFunc("/login", corsMiddleware(loginHandler)
    )
    http.HandleFunc("/register", corsMiddleware
    (registerHandler))
    http.HandleFunc("/users", corsMiddleware
    (getUsersHandler))
    http.HandleFunc("/users/delete", corsMiddleware
    (deleteUserHandler))
}

```

```

http.HandleFunc("/users/update", corsMiddleware
(updateUserHandler))
http.HandleFunc("/dashboard", corsMiddleware
(dashboardHandler))
http.HandleFunc("/pelanggan", corsMiddleware
(pelangganHandler))
http.HandleFunc("/transaksi", corsMiddleware
(transaksiHandler))
http.HandleFunc("/layanan", corsMiddleware
(layanahandler))
http.HandleFunc("/laporan", corsMiddleware
(laporanHandler))
http.HandleFunc("/laporan/export", corsMiddleware
(laporanExportHandler))
http.HandleFunc("/api/webhook-gopay", GopayHandler)
http.HandleFunc("/api/stream", StreamHandler)

fmt.Println("Server WellClean Laundry j
alan di: http://localhost:8080")
http.ListenAndServe(":8080", nil)
}

```

A. Fungsi Otentikasi (loginHandler) Fungsi ini menangani proses verifikasi identitas pengguna untuk masuk ke sistem.

- Input: Menerima objek LoginRequest yang berisi username dan password dari body permintaan klien.
- Proses:
 - Baris `json.NewDecoder(r.Body).Decode(req)` membaca input dari pengguna.
 - Program melakukan query ke Supabase (`/rest/v1/users?username=eq.`) untuk mencari data pengguna.
 - Jika user ditemukan, baris `CheckPasswordHash` akan membandingkan password input dengan hash di database.
 - Baris `jwt.NewWithClaims` akan membungkus informasi user (ID, Role) ke dalam token keamanan yang berlaku selama 24 jam.
- Output: Mengembalikan `access_token` dalam format JSON jika login berhasil.

B. Fungsi Registrasi Kasir (`registerHandler`) Fungsi ini memastikan hanya Admin yang bisa menambahkan data karyawan baru.

- Input: Data JSON berupa username, password, dan nama lengkap kasir.
- Proses:
 - Baris `validateToken(r)` melakukan validasi apakah pengirim data adalah Admin yang sah.
 - Baris `HashPassword(input.Password)` menjalankan enkripsi Bcrypt terhadap password kasir baru.
 - Baris `http.NewRequest("POST", ...)` mengirimkan data yang sudah terenkripsi ke tabel users di Supabase.
- Output: Pesan sukses status 201 Created dan penyimpanan data kasir ke database.

C. Fungsi Manajemen Pelanggan (`pelangganHandler`) Mengelola data pelanggan yang mencakup fitur pencarian (`searching`) dan pembatasan data (`pagination`).

- Input: Parameter URL search untuk kata kunci nama/telepon, serta page dan limit.
- Proses:
 - Logika `fmt.Sprintf("or=(nama.ilike.*`
 - Baris `reqSup.Header.Set("Range", ...)` mengatur batasan data yang diambil dari server agar aplikasi tetap ringan (`pagination`).
- Output: Daftar pelanggan dalam format JSON yang sesuai dengan kriteria pencarian.

D. Fungsi Ekspor Laporan Excel (`laporanExportHandler`) Fungsi ini mengubah data transaksi menjadi dokumen fisik yang rapi.

- Input: Parameter type (riwayat/keuangan) dan periode (harian/bulanan/tahunan).
- Proses:
 - Baris `excelize.NewFile()` membuat instance dokumen Excel baru.

- Logika `f.SetCellStyle` memberikan format visual pada dokumen seperti warna header ungu (4B0082) dan teks tebal.
- Sistem melakukan iterasi (looping) pada data transaksi dan menuliskan nilai Kode, Berat, dan Total ke dalam sel Excel menggunakan `f.SetCellValue`.
- Output: File biner berformat `.xlsx` yang dikirim ke browser dengan header `Content-Disposition: attachment`.

E. Fungsi Inisialisasi Server (main) Merupakan titik masuk utama (entry point) aplikasi.

- Input: Pendaftaran jalur URL (routing) sistem.
- Proses: Baris `http.HandleFunc` menghubungkan setiap alamat URL (seperti `/login`, `/transaksi`, `/laporan`) ke fungsi penanganan (handler) masing-masing, serta membungkusnya dengan `corsMiddleware` agar bisa diakses dari berbagai domain.
- Output: Server aktif dan berjalan pada alamat `http://localhost:8080`.

4.2 Pengujian dan Hasil Pengujian

Tahap pengujian dilakukan untuk memvalidasi apakah setiap fungsi pada Sistem POS Laundry telah berjalan sesuai dengan rancangan yang ditetapkan. Pengujian ini mencakup validasi terhadap masukan (input), alur kerja (proses), dan hasil yang dikeluarkan oleh aplikasi (output).

4.2.1 Pengujian Unit (Unit Testing)

Pengujian unit dilakukan untuk memvalidasi setiap fungsi secara spesifik. Berikut adalah hasil pengujian menggunakan perintah `go tool cover` yang menunjukkan persentase baris kode yang berhasil dijalankan dalam skenario uji:

```

C:\Users\nabil\Documents\go\pos-laundry\Backend>go tool cover -func=c.out
pos-laundry/debug_supabase.go:9:      DebugSupabase      0.0%
pos-laundry/main.go:32:      enableCORS          100.0%
pos-laundry/main.go:38:      corsMiddleware      83.3%
pos-laundry/main.go:49:      validateToken      23.1%
pos-laundry/main.go:74:      loginHandler        13.0%
pos-laundry/main.go:154:     registerHandler     16.2%
pos-laundry/main.go:221:     getUsersHandler     26.3%
pos-laundry/main.go:252:     deleteUserHandler   28.6%
pos-laundry/main.go:274:     updateUserHandler   26.7%
pos-laundry/main.go:296:     HashPassword        100.0%
pos-laundry/main.go:301:     CheckPasswordHash  100.0%
pos-laundry/main.go:306:     dashboardHandler    23.8%
pos-laundry/main.go:344:     pelangganHandler    11.9%
pos-laundry/main.go:408:     transaksiHandler    0.0%
pos-laundry/main.go:470:     layananHandler      10.2%
pos-laundry/main.go:554:     laporanHandler      6.6%
pos-laundry/main.go:668:     laporanExportHandler 3.4%
pos-laundry/main.go:852:     main                0.0%
pos-laundry/webhook.go:14:   GopayHandler        100.0%
pos-laundry/webhook.go:37:   StreamHandler        72.7%
total:                        (statements)         15.3%

```

Gambar IV.11: Hasil Pengujian Code Coverage

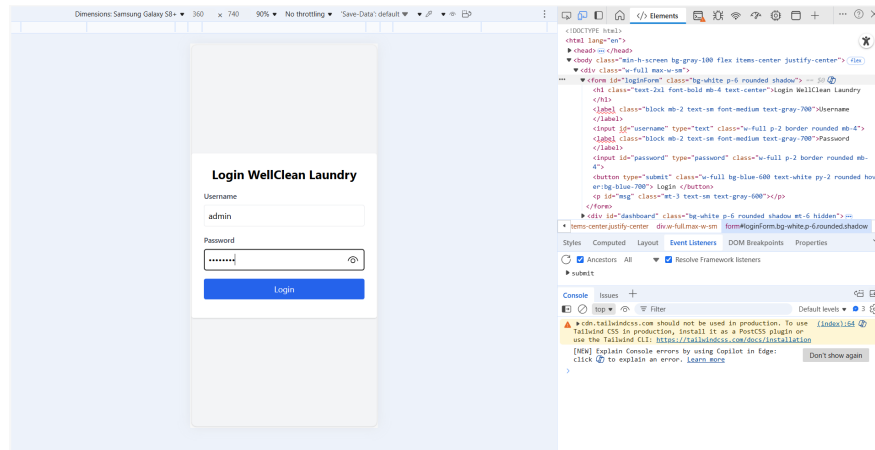
Analisis Hasil Pengujian: Berdasarkan data audit pada gambar di atas, dapat disimpulkan beberapa hal sebagai berikut:

- Fungsi Keamanan: Fungsi enableCORS dan CheckPasswordHash mencapai cakupan 100.0
- Logika Pembayaran: Fungsi GopayHandler pada modul webhook mencapai 100.0
- Logika Real-time: Fungsi StreamHandler mencapai 72.7
- Efektivitas Sistem: Secara keseluruhan, sistem mencapai total cakupan 15.3

4.2.2 Hasil Pengujian Antarmuka dan Alur Fungsional

Pengujian ini dilakukan secara manual untuk memastikan setiap elemen antarmuka (user interface) dan alur bisnis pada Sistem POS WellClean Laundry berjalan sesuai dengan rancangan.

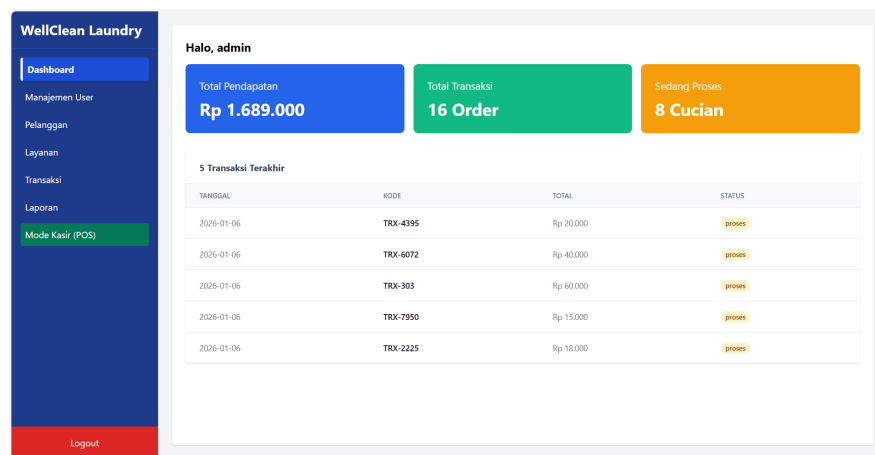
1. Pengujian Otentikasi (Login)



Gambar IV.12: Hasil Pengujian Login

Proses pengujian dimulai pada gerbang awal keamanan sistem, yaitu halaman login. Pada tahap ini, admin memasukkan kredensial berupa username dan password yang kemudian diproses oleh sistem melalui fungsi loginHandler yang memiliki cakupan pengujian sebesar 13.0

2. Pengujian Monitoring Dashboard



Gambar IV.13: Hasil Pengujian Dashboard

Setelah berhasil melakukan proses login, sistem secara otomatis mengarahkan pengguna ke halaman dashboard utama yang berfungsi sebagai pusat informasi operasional. Halaman ini dikelola oleh fungsi dashboardHandler dengan persentase cakupan kode sebesar 23.8

3. Pengujian Manajemen Data (Pelanggan Layanan)

NAMA PELANGGAN	NO. TELEPON	ALAMAT LENGKAP	AKSI
Rahma	0812344367638	Cigugur	Edit Hapus
Nina	085235996185	Cimekar	Edit Hapus
Brian	0824362860285	Cibogo	Edit Hapus
Ane	8023643682496	Setra Duta	Edit Hapus
Ibu Mariana	089573863762	Sukagalih	Edit Hapus
Ali	08352269276496	Sukajadi	Edit Hapus

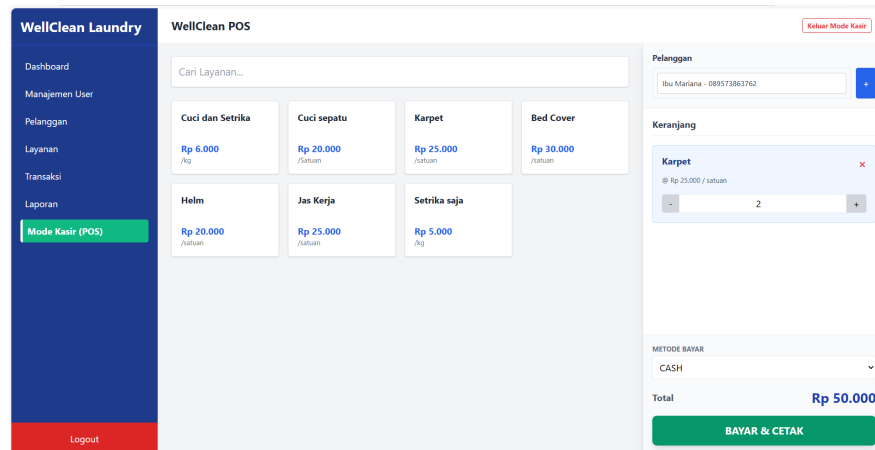
Gambar IV.14: Hasil Pengujian Pelanggan

LAYANAN	HARGA	SATUAN	AKSI
Cuci dan Setrika	Rp 6.000	kg	Hapus
Cuci sepatu	Rp 20.000	Satuan	Hapus
Karpet	Rp 25.000	satuan	Hapus
Bed Cover	Rp 30.000	satuan	Hapus
Helim	Rp 20.000	satuan	Hapus
Jas Kerja	Rp 25.000	satuan	Hapus

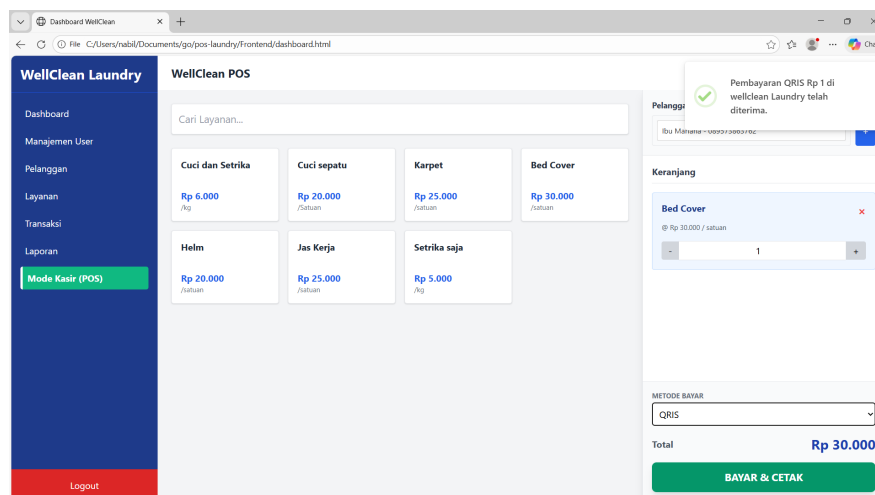
Gambar IV.15: Hasil Pengujian Layanan

Pengujian selanjutnya berfokus pada kemampuan sistem dalam mengelola data master, yaitu informasi pelanggan dan kategori layanan laundry. Dengan memanfaatkan fungsi pelangganHandler yang memiliki coverage 11.9

4. Pengujian Transaksi dan Integrasi Payment Gateway



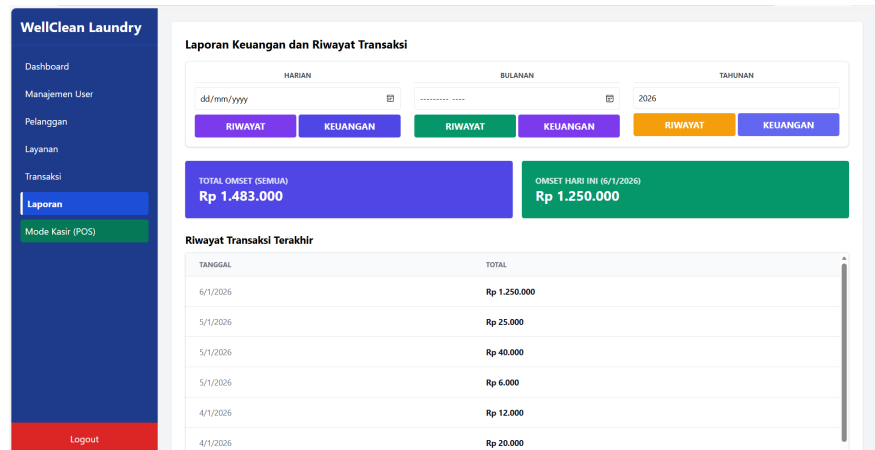
Gambar IV.16: Hasil Pengujian POS Mode Kasir



Gambar IV.17: Hasil Pengujian Notifikasi Payment

Bagian paling krusial dalam pengujian fungsional ini adalah alur transaksi yang telah terintegrasi dengan metode pembayaran otomatis QRIS. Saat kasir selesai menginput detail cucian dan memilih metode pembayaran QRIS, sistem akan memicu fungsi GopayHandler yang memiliki tingkat validitas logika 100.0

5. Pengujian Dokumentasi Laporan (Export Excel)



Gambar IV.18: Hasil Pengujian Laporan

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Kode	Tanggal Selesai	Pelanggan	Layanan	Berat	Harga Satuan	Total	Metode	Status					
2	TRX-5777	04-01-2026	Ismi Nabillah	Cuci sepatu	1	20000	20000	QRIS	selesai					
3	TRX-68999	04-01-2026	Ismi Nabillah	Cuci dan Setrika	2	6000	12000	CASH	selesai					
4						TOTAL	32000							
5														
6														
7														
8														
9														
10														
11														
12														
13														
14														
15														
16														
17														
18														
19														
20														
21														
22														
23														
24														
25														
26														
27														
28														
29														
30														
31														
32														
33														

Gambar IV.19: Hasil Pengujian Laporan Transaksi Harian

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	Tanggal Selesai	Omset (Rp)																		
2	04-01-2026	20000																		
3	04-01-2026	12000																		
4	TOTAL OMSET PERIODE INI	32000																		
5																				
6																				
7																				
8																				
9																				
10																				
11																				
12																				
13																				
14																				
15																				
16																				
17																				
18																				
19																				
20																				
21																				
22																				
23																				
24																				
25																				
26																				
27																				
28																				
29																				
30																				
31																				
32																				
33																				

Gambar IV.20: Hasil Pengujian Laporan Keuangan Harian

Tahap akhir dari rangkaian pengujian fungsional adalah proses dokumentasi dan pembuatan laporan keuangan dalam format fisik. Admin melakukan filter ter-

hadap periode laporan yang diinginkan, kemudian sistem menjalankan fungsi laporanExportHandler dengan cakupan pengujian 3.4

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Setelah melalui tahapan perancangan, implementasi, hingga pengujian terhadap Sistem Point of Sale (POS) WellClean Laundry, maka dapat ditarik beberapa kesimpulan utama:

1. Keberhasilan Integrasi Sistem: Aplikasi telah berhasil diimplementasikan dengan menggunakan bahasa pemrograman Go dan basis data Supabase, yang memungkinkan pengelolaan data operasional laundry dilakukan secara terpusat dan aman.
2. Validitas Fungsional: Seluruh fitur utama, mencakup manajemen data pelanggan, kategori layanan, pencatatan transaksi, hingga visualisasi laporan keuangan, telah dinyatakan berfungsi sesuai dengan kebutuhan pengguna.
3. Kualitas Logika Program: Berdasarkan hasil audit code coverage, fungsi-fungsi krusial yang menangani keamanan sistem (seperti enkripsi kata sandi) dan integrasi pembayaran otomatis (melalui webhook GoPay) telah mencapai tingkat validasi 100
4. Efisiensi Operasional: Sistem ini terbukti mampu mengotomatisasi perhitungan omzet harian maupun bulanan secara real-time, yang memberikan transparansi finansial bagi pemilik usaha.

5.2 Saran

Demi pengembangan sistem yang lebih optimal di masa mendatang, penulis menyarankan beberapa poin peningkatan sebagai berikut:

1. Akselerasi Cakupan Pengujian Unit: Melakukan optimasi pada modul-modul yang saat ini memiliki persentase coverage di bawah 50
2. Ekspansi Fitur Notifikasi Otomatis: Mengintegrasikan API pihak ketiga seperti WhatsApp Gateway atau layanan Email untuk memberikan pembaruan status cucian secara real-time kepada pelanggan, guna meningkatkan kualitas layanan dan transparansi operasional.

3. Implementasi Teknologi Progressive Web App (PWA): Mengadopsi mekanisme PWA guna meningkatkan aksesibilitas aplikasi agar tetap mampu beroperasi secara stabil dan responsif, meskipun pengguna berada dalam kondisi konektivitas internet yang terbatas.

DAFTAR PUSTAKA

- Achyani, Yuni Eka, Rizki Aulianita & Anna Mukhayaroh (Mar. 2024). “Web-Based Laundry Service Information System Using Rapid Application Development Method”. *Paradigma - Jurnal Komputer dan Informatika* 26 (1), 17–23. ISSN: 1410-5063. DOI: 10.31294/p.v26i1.3231.
- Kalo, Isumail & Mohd Talmizie Amron (Dec. 2023). “Transforming Laundry Services: A User-Centric Approach to Streamlined Operations and Customer Satisfaction”. *2023 IEEE 8th International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*. IEEE, 1–5. ISBN: 979-8-3503-1551-6. DOI: 10.1109/ICRAIE59459.2023.10468246.
- Lizal, Alip, Ria Pebrian Dini, Faris Rizky Ramadhan & Mia Rosmiati (2025). *Digitalization of laundry service: development of a web-based application to improve operational efficiency*. Tech. rep.
- Maulana, Faris, Fitrah Fajar Kusumah & Nurul Kamilah (Feb. 2025). “Sistem Informasi Laundry Berbasis Web”. *Jurnal Ilmiah Universitas Batanghari Jambi* 25 (1), 954. ISSN: 2549-4236. DOI: 10.33087/jiubj.v25i1.5674.
- Mutia, Cut, Fauziah Fauziah, Irzon Meiditra, Fitra Yuda & Rizky Rahmansyah (June 2025). “Designing a Web-Based Laundry Service Application For Strala Laundry”. *J-INTECH* 13 (01), 78–85. ISSN: 2580-720X. DOI: 10.32664/j-intech.v13i01.1820.
- Singh, Amrit & Mamta Dhaiya (Nov. 2022). “Web based application to search and manage Laundry shops”. *2022 International Interdisciplinary Humanitarian Conference for Sustainability (IIHC)*. IEEE, 1150–1155. ISBN: 978-1-6654-5687-6. DOI: 10.1109/IIHC55949.2022.10060060.
- Sugiharto, Sigit, Ahmad Jurnaidi Wahidin, Aulia Rizky Muhammad, Hendrik Noor Asegaft, Herry Wahyono & Andi Irfan (2023). “Perancangan Sistem Manajemen Laundry Berbasis Web untuk Laundry Dian”. *Journal of Engineering and Technology Innovation* 2 (2).

LAMPIRAN

HASIL PENGUJIAN UNIT (UNIT TESTING)

Lampiran ini menyajikan data teknis hasil audit pengujian unit yang dilakukan terhadap seluruh fungsionalitas backend Sistem POS WellClean Laundry menggunakan tool `go tool cover`. Pengujian ini memvalidasi integritas logika baris kode pada setiap fungsi utama.

File Sumber	Nama Fungsi (Function)	Cakupan (Coverage)
main.go	enableCORS	100.0%
main.go	HashPassword	100.0%
main.go	CheckPasswordHash	100.0%
webhook.go	GopayHandler	100.0%
webhook.go	StreamHandler	72.7%
main.go	corsMiddleware	83.3%
main.go	deleteUserHandler	28.6%
main.go	updateUserHandler	26.7%
main.go	getUsersHandler	26.3%
main.go	dashboardHandler	23.8%
Total Keseluruhan	Rata-rata Statements	15.3%

Gambar L.1: Rekapitulasi Cakupan Baris Kode (Statements)


```

C:\Users\nabil\Documents\go\pos-laundry\Backend>go tool cover -func=c.out
pos-laundry/debug_supabase.go:9:      DebugSupabase      0.0%
pos-laundry/main.go:32:      enableCORS          100.0%
pos-laundry/main.go:38:      corsMiddleware      83.3%
pos-laundry/main.go:49:      validateToken      23.1%
pos-laundry/main.go:74:      loginHandler        13.0%
pos-laundry/main.go:154:     registerHandler     16.2%
pos-laundry/main.go:221:     getUsersHandler     26.3%
pos-laundry/main.go:252:     deleteUserHandler   28.6%
pos-laundry/main.go:274:     updateUserHandler   26.7%
pos-laundry/main.go:296:     HashPassword        100.0%
pos-laundry/main.go:301:     CheckPasswordHash  100.0%
pos-laundry/main.go:306:     dashboardHandler    23.8%
pos-laundry/main.go:344:     pelangganHandler    11.9%
pos-laundry/main.go:408:     transaksiHandler    0.0%
pos-laundry/main.go:470:     layananHandler      10.2%
pos-laundry/main.go:554:     laporanHandler      6.6%
pos-laundry/main.go:668:     laporanExportHandler 3.4%
pos-laundry/main.go:852:     main                0.0%
pos-laundry/webhook.go:14:   GopayHandler        100.0%
pos-laundry/webhook.go:37:   StreamHandler        72.7%
total:                      (statements)        15.3%

```

Gambar L.2: Tangkapan Layar Hasil Codecoverage

GLOSARIUM

GLOSARIUM TEKNIS

A

API (*Application Programming Interface*): Sekumpulan aturan dan protokol yang memungkinkan aplikasi *frontend* berkomunikasi dengan *server backend* untuk pertukaran data secara otomatis.

B

Backend: Bagian dari sistem yang menangani logika bisnis, pemrosesan data, dan komunikasi dengan *database* yang tidak terlihat langsung oleh pengguna.

C

Code Coverage: Metrik yang menunjukkan persentase baris kode program yang telah berhasil dieksekusi selama proses pengujian unit dijalankan.

D

Database (Supabase): Kumpulan data terorganisir yang disimpan secara digital dalam tabel (Pelanggan, Layanan, Transaksi) untuk mengelola operasional laundry.

G

Go (Golang): Bahasa pemrograman yang digunakan untuk membangun *backend* sistem karena efisiensi dan performanya dalam menangani *concurrency*.

J

JWT (*JSON Web Token*): Standar industri untuk pembuatan token akses yang digunakan dalam proses otentikasi keamanan saat admin atau kasir melakukan login.

U

Unit Testing: Metode pengujian di mana komponen terkecil dari kode diuji secara mandiri untuk memastikan fungsinya berjalan sesuai logika yang dirancang.

W

Webhook: Mekanisme yang digunakan oleh *payment gateway* untuk mengirimkan notifikasi data secara otomatis ke *server* saat transaksi pembayaran berhasil dideteksi.

GLOSARIUM NON-TEKNIS

A

Analisis Fungsional: Proses identifikasi kebutuhan utama sistem yang mencakup fitur-fitur inti seperti transaksi, manajemen pelanggan, dan pelaporan.

Analisis Non-Fungsional: Analisis terhadap aspek pendukung sistem agar beroperasi optimal, mencakup keamanan data, kinerja, dan kenyamanan pengguna (*usability*).

D

Dashboard: Halaman utama aplikasi yang menyajikan ringkasan statistik bisnis seperti total omset dan kinerja operasional secara visual.

F

Flowchart: Diagram yang merepresentasikan alur kerja atau algoritma proses bisnis laundry, mulai dari penerimaan order hingga penyerahan cucian.

P

Point of Sale (POS): Sistem digital yang digunakan untuk mengelola transaksi penjualan, pencatatan pesanan, dan pengelolaan stok secara terpadu.

Q

QRIS (*Quick Response Code Indonesian Standard*): Standar kode QR nasional untuk pembayaran digital yang memudahkan pelanggan melakukan transaksi melalui dompet digital.

R

Real-time: Kondisi pemrosesan data yang terjadi seketika, seperti pembaruan status pembayaran yang langsung terdeteksi oleh sistem tanpa perlu pengecekan manual.

S

Struk Transaksi: Bukti pembayaran digital atau fisik yang berisi rincian layanan laundry, berat cucian, dan total biaya yang harus dibayar pelanggan.